

Search Lab

Architecture Reference

Disposable Solr / Elasticsearch / OpenSearch clusters, synthetic and real data, open-loop load testing, chaos engineering, live tuning, semantic search, and a single-page control panel — all driven from one CLI.

Package	searchlab · 6,521 lines across 31 modules
Frontend	searchlab/templates/dashboard.html · 2,096 lines, no build step
Tests	23 files, 216 tests, no Docker required
Repository	github.com/patrykm10/searchlab (private)
Engines	Solr 9.x (primary) · Elasticsearch · OpenSearch

Contents

- [1](#) What Search Lab Is
- [2](#) High-Level Architecture
- [3](#) Repository Layout
- [4](#) The searchlab Package — Module Reference
 - [4.1](#) CLI & Cluster Lifecycle
 - [4.2](#) Engine Abstraction (Solr / Elasticsearch / OpenSearch)
 - [4.3](#) Data: Generation, Real-File Ingestion, Schema
 - [4.4](#) Open-Loop Load Testing Engine
 - [4.5](#) Query Builder & Semantic Search
 - [4.6](#) Live Tuning
 - [4.7](#) Vector Search & Embeddings
 - [4.8](#) Observability: Metrics, Insights, Logs, Segments
 - [4.9](#) Chaos, Drills & Sweeps
 - [4.10](#) Reporting & Interop
- [5](#) The Control Panel (Dashboard)
 - [5.1](#) Server Architecture
 - [5.2](#) API Route Reference
 - [5.3](#) Frontend: One HTML File, No Build Step
- [6](#) Data Flow Walkthroughs
- [7](#) Cluster State & Configuration
- [8](#) CLI Command Reference
- [9](#) Testing Strategy
- [10](#) Design Principles

1. What Search Lab Is

Search Lab is a CLI and web control panel for spinning up disposable search-engine clusters, loading them with data of controllable shape (synthetic or real), and finding out how they actually behave — under load, under failure, under tuning changes — rather than how they're assumed to behave. It targets two jobs: **learning** how a search engine's internals work by causing things and watching them happen, and **reproducing production incidents** on demand with a deterministic seed.

It began as a Solr-only tool (hence the package name) and grew an engine-abstraction layer that also drives Elasticsearch and OpenSearch through the same commands, the same dashboard, and the same report format. Everything downstream of engine selection — load testing, chaos, metrics, the control panel — is written against a small internal interface (`engines.py`) rather than against Solr's API directly.

The project has two front doors that stay in sync: a scriptable CLI (`searchlab <command>`) for reproducible, automatable work, and a single-page web control panel (`searchlab dashboard`) for driving the same underlying actions interactively — start a load test, tune a cache size, kill a node, run a query — with live feedback.

2. High-Level Architecture

Three layers, each replaceable without touching the others:

Layer	Lives in	Talks to
CLI	<code>cli.py</code> (Click commands)	The engine layer, directly and synchronously
Engine abstraction	<code>engines.py</code> + <code>cluster.py</code>	Docker Compose (cluster lifecycle) and each engine's HTTP API (indexing, querying, admin)
Control panel	<code>dashboard.py</code> + <code>actions.py</code> + <code>templates/dashboard.html</code>	The same engine layer, via a background asyncio loop and a JSON API the browser polls

Cluster state — which engine, how many nodes, heap size, which port each node listens on — is a small JSON file (`.searchlab/spec.json`) written by `searchlab up` and read by every subsequent command. There is no server-side database: the cluster itself, plus that one spec file, is the entire state a session needs. Tearing down is `searchlab down` (add `--volumes` to also discard indexed data).

The dashboard is not a separate service with its own persistence — it is a thin, stateless-between-restarts HTTP server that reads the same `.searchlab/` directory and drives the same functions the CLI calls. Restarting it loses only in-memory state (a running load test, a loaded embedding model, collapsed-section preferences on the client); the cluster and its data are unaffected.

3. Repository Layout

Path	Contents
<code>searchlab/</code>	The Python package — 31 modules, described in full in Section 4
<code>searchlab/templates/</code>	<code>dashboard.html</code> (the control panel UI) and the two <code>docker-compose.yml.j2</code> Jinja templates (Solr, ES/OS) rendered per cluster
<code>searchlab/lessons/</code>	YAML lesson scripts for <code>`searchlab learn`</code> (cluster-anatomy, leader-election, commits-and-visibility)
<code>profiles/</code>	YAML data-shape profiles: <code>default.yaml</code> (e-commerce-ish), <code>high-cardinality.yaml</code> , <code>vectors.yaml</code>
<code>queries/</code>	YAML query-template mixes for load testing: <code>default</code> , <code>complex</code> , <code>es-default</code> , and per-engine vector-search variants
<code>examples/</code>	Sample chaos scenarios, drill definitions, and a heap sweep matrix
<code>tests/</code>	23 files, 216 tests — see Section 9
<code>.searchlab/</code>	Generated at runtime: <code>docker-compose.yml</code> , <code>spec.json</code> , <code>live-load.json</code> , <code>gc-logs/</code> — never committed
<code>pyproject.toml</code>	Package metadata. Core deps: <code>click</code> , <code>httpx</code> , <code>jinja2</code> , <code>pyyaml</code> . Optional extras: <code>[recall]</code> adds <code>numpy</code> , <code>[embed]</code> adds <code>fastembed</code>

4. The searchlab Package — Module Reference

Modules are grouped below by what they're for, not alphabetically. Line counts are from the current tree and give a rough sense of surface area, not importance — `actions.py` is large because it's the seam between every backend capability and the dashboard's HTTP routes, not because any one thing it does is complex.

4.1 CLI & Cluster Lifecycle

Module	Lines	Responsibility
<code>cli.py</code>	694	Every Click command — the full list is Section 8. Thin wrappers: each command parses flags, loads or builds a <code>ClusterSpec</code> , and calls straight into the modules below.
<code>cluster.py</code>	297	<code>ClusterSpec</code> dataclass; renders and runs <code>docker-compose</code> (<code>up/down/wait_healthy</code>); <code>create/delete/reload/split</code> a collection; <code>commit</code> , <code>optimize</code> (<code>merge</code>), <code>expunge</code> deletes, <code>purge</code> all documents; <code>add/remove</code> a replica; <code>cluster_overview()</code> for the health snapshot.
<code>gates.py</code>	97	Parses human-friendly durations (" <code>90s</code> ", " <code>2m</code> ", " <code>1h30m</code> ") and counts (" <code>10k</code> ", " <code>1.5m</code> "), and evaluates CI-style regression assertions (<code>--assert "p99_ms<80"</code>).

4.2 Engine Abstraction

`engines.py` (250 lines) is the interface every other module codes against instead of calling Solr directly. Each engine (`SolrEngine`, `ElasticsearchEngine`, `OpenSearchEngine`) implements the same methods: `base_url`, `health_ok`, `create_index / delete_index`, `bulk_request` (indexing payload shape), `search_request` (query shape), `cluster_overview`, `snapshot_node` (metrics normalization), and `default_queries()`. Adding a fourth engine means implementing this one class; nothing in `loadtest.py`, `dashboard.py`, or `chaos.py` needs to know it exists.

4.3 Data: Generation, Real-File Ingestion, Schema

Module	Lines	Responsibility
<code>datagen.py</code>	140	Generates synthetic JSONL from a YAML profile. Field types: text, keyword, categorical (with Zipf skew), multivalued, int, float, date, bool, id, vector (clustered, not uniform-random — real embeddings cluster, and uniform vectors are pathological for ANN).
<code>tabular.py</code>	214	Indexes real CSV / TSV / JSON files. Sniffs the format, infers a type per column from a sample, and renames columns onto Solr's dynamic-field suffixes (<code>price -> price_f</code> , <code>title -> title_s</code> or <code>title_t</code> depending on text length). <code>--dry-run</code> shows the mapping before anything is sent.
<code>indexer.py</code>	128	Async bulk indexing with worker threads and batching; shared by synthetic, tabular, and dashboard-triggered indexing.

Module	Lines	Responsibility
<code>schema.py</code>	207	Derives explicit Solr fields (and DenseVectorField fieldTypes for vector profiles) from a data profile, for when dynamic fields aren't precise enough for the repro.
<code>configset.py</code>	95	Installs a lab-owned Solr configset on first collection creation: a copy of <code>_default</code> with <code>indexConfig</code> parameterized via <code>\${searchlab.*}</code> properties, which is what makes merge-policy knobs (Section 4.6) live-tunable at all.

4.4 Open-Loop Load Testing Engine

`loadtest.py` (386 lines) is the heart of the load-testing side. It is deliberately **open-loop**: requests fire on a fixed wall-clock schedule derived from the target rate, regardless of whether earlier requests have returned. A closed-loop tool (wait for a response, then send the next one) automatically throttles itself when the server slows down, which hides exactly the worst latencies an incident produces — a well-known measurement bug called *coordinated omission*. When the client's in-flight cap is hit, scheduled requests are counted as dropped rather than silently queued or sent late.

A `LoadControl` object makes a run steerable while it executes: its `rps` field is read on every scheduler tick (so the dashboard's live RPS slider works against a running test) and its `stop_requested` flag ends the run early. Query templates support `{RAND_WORD}`, `{RAND_INT:lo:hi}`, and `{RAND_VECTOR:dims}` placeholders substituted per request, so result caches can't flatter the numbers. Reports are JSON with a per-5-second timeline (`rps`, `p50`, `p99`, `errors`) for spotting commit/merge sawtooth patterns.

4.5 Query Builder & Semantic Search

`query.py` (173 lines) backs the dashboard's interactive query section. `build_params()` turns a form-shaped dict into Solr query params: parser selection (lucene / dismax / edismax), `qf` field weights, repeated `fq` filters, faceting, and — when semantic mode is on — embeds the query text and rewrites it into a `{!knn}` clause instead of a keyword query. `run_query()` executes it and normalizes the response: hit count, `QTime`, flattened facet buckets, the exact request URL (shown in the UI so it teaches Solr's real API), and — with `explain on` — the parsed query, per-component timing, and score explanations.

4.6 Live Tuning

`tuning.py` (249 lines) exposes real Solr settings as UI sliders that take effect within seconds, no restart. Two mechanisms depending on the setting: settings on Solr's Config-API whitelist (commit intervals, cache sizes, autowarm counts) use `set-property` directly. Merge-policy and indexing-buffer settings — `segmentsPerTier`, `maxMergedSegmentMB`, `deletesPctAllowed`, `ramBufferSizeMB` — are not on that whitelist, so the lab configset (4.3) parameterizes them as `${searchlab.*}` properties and `set-user-property` writes them live. Both paths write an overlay into ZooKeeper and trigger an automatic core reload; `solrconfig.xml` on disk is intentionally never touched, which is why each knob also exposes a “verify” link straight to the Solr config endpoint that proves the value actually changed.

4.7 Vector Search & Embeddings

Module	Lines	Responsibility
embeddings.py	104	Wraps fastembed (ONNX-based — ~158 MB vs. ~2.5 GB for the sentence-transformers/torch route, for the same models). Model registry with dimensions (MiniLM 384d, BGE small/base, Nomic 768d); models load lazily and are cached in-process.
vectorize.py	93	Adds embeddings to documents already indexed: reads them back via cursorMark streaming, embeds a chosen text field, and writes the vector back with an atomic partial update — the same shape as a real re-embedding migration.
recall.py	152	Measures recall@k for ANN search: brute-force ground truth via numpy against the engine's approximate kNN results, across a candidate-count sweep, to find where recall actually falls off.

Synthetic vector profiles (clustered random points) remain the tool for load-testing ANN behavior at scale; embeddings.py is for testing whether search finds the *right* documents, which random vectors have no meaningful answer to.

4.8 Observability: Metrics, Insights, Logs, Segments

Module	Lines	Responsibility
metrics.py	203	Polls each node's /admin/metrics and normalizes JVM heap/GC, per-core doc counts, cache hit ratios, update/merge counters, segment count and size, and Lucene searcher internals (maxDoc, searchers opened, warmup time, full-vs-skipped sort counts) into one snapshot shape shared by the CLI and dashboard.
insights.py	211	A small rules engine (InsightsEngine) that turns raw snapshot numbers into plain-language findings: heap thresholds, GC-time share, commit-rate storms (from snapshot-to-snapshot deltas), low cache hit ratios, high deleted-document share, missing nodes, non-green collection health, and load-test saturation / dropped requests. Each finding carries a severity and a why-it-matters sentence, not just a number.
segments.py	92	Reads one replica's /admin/segments: per-segment size, live/deleted doc counts, and — the useful part — whether each segment came from a flush or a merge, which is what tells you whether the merge policy is keeping up.
loglex.py	219	Two jobs on the raw docker-compose log stream: classify() tags recognizable events (commit, new searcher, merge, leader election, recovery) with a plain-language description; parse_request() extracts structured query/index traffic from Solr's request log, filtering out internal shard fan-out and leader-to-replica replication so nothing is double-counted.
logstream.py	150	Tails `docker compose logs --follow` in a daemon thread into two bounded ring buffers (activity, traffic). Drops multi-line JSON continuation fragments (Overseer state dumps) from the activity feed unless they follow a WARN/ERROR, where they're the stack trace that matters.

4.9 Chaos, Drills & Sweeps

Module	Lines	Responsibility
chaos.py	106	Fault injection on individual nodes: kill (SIGKILL), pause (SIGSTOP — simulates a frozen JVM / GC pause), unpause, start, restart. Also runs a timed scenario file standalone.
drill.py	132	Orchestrates load + chaos + before/after metrics from one YAML file into a single annotated HTML report — the one-command version of a manual failure drill.
sweep.py	153	Runs one workload across a config matrix (heap, GC flags, node count, engine, ...), a fresh cluster per cell, and produces a comparison table with the best cell per metric highlighted.

4.10 Reporting & Interop

Module	Lines	Responsibility
report.py	334	JSON report save/load, text and HTML comparison between two runs (version regressions, A/B configs), dependency-free HTML rendering with inline SVG charts.
gclog.py	110	Parses JDK unified-logging GC logs: pause counts and tail percentiles per pause type, throughput lost to GC, Full GC detection.
replay.py	191	Replays a real Solr request log (or a plain query-per-line file) at original, scaled, or fixed pacing — turns "what happened last Tuesday" into a repeatable load test.
k8s.py	117	Exports the idiomatic Kubernetes operator manifest for the current or a specified engine (Solr Operator / ECK / OpenSearch Operator).
explain.py	107	Runs a query with debug=true and translates Solr's debug block into three plain-language stories: what the query became after analysis, where time went per component, and why the top hit scored what it did.
learn.py	182	Runs interactive lesson scripts (searchlab/lessons/*.yaml) against the live cluster — a lesson's "wait" step polls real cluster state after telling you to go cause something (e.g. kill a node) in another terminal.

5. The Control Panel (Dashboard)

5.1 Server Architecture

dashboard.py (320 lines) runs a stdlib `ThreadingHTTPServer` — one thread per HTTP request — with three long-lived collaborators constructed once at startup and shared across every request:

Component	Lives in	Role
ActionRunner	actions.py (615 lines)	Owns a dedicated background asyncio event loop (its own thread) and every piece of mutable session state: the running load test's <code>LoadControl</code> , the loaded embedding model, in-flight indexing/maintenance jobs, and a rolling action-outcome log. Request-handler threads submit coroutines to this loop with <code>run_coroutine_threadsafe</code> rather than awaiting them directly, which is what lets the RPS slider mutate a load test that's already running.
LogStream	logstream.py	One daemon thread tailing container logs into the two ring buffers described in 4.8. A <code>FakeLogStream</code> subclass synthesizes plausible lines for <code>--demo</code> mode.
InsightsEngine	insights.py	Stateful across polls (it needs the previous snapshot to compute rates like <code>commits/minute</code>), so one instance lives for the server's lifetime and is handed each new snapshot as it's built.

Every write-side route follows the same shape: validate inputs before touching the network, reject with a clear message if a conflicting job is already running (one load test, one indexing job, one maintenance operation at a time), submit the real work to the `ActionRunner`, and return immediately with `{"ok": true}`. The browser learns the outcome on its next poll of `/api/snapshot`, not from the POST response — which is why the UI stays responsive even while a merge or an embedding pass is running in the background.

5.2 API Route Reference

All routes are same-origin JSON, served from and consumed only by `dashboard.html` — there is no external API contract to keep stable.

Route	Purpose
GET /api/snapshot	The core poll: cluster health, per-node metrics, load-test live stats, action-runner state, and computed insights — everything the page's charts and status lines render from
GET /api/logs , /api/traffic	Incremental (<code>since=seq</code>) reads from the two <code>LogStream</code> ring buffers
GET /api/topology , /api/tuning , /api/fields	Collection shard/replica layout; current tuning-knob values; indexed field names and types (from Luke) for the query builder's pickers

Route	Purpose
GET /api/segments	Per-replica Lucene segment detail (Section 4.8)
GET /api/model	Embedding-model load state
POST /api/load/start, /rps, /stop	Start, live-retarget, or stop the running load test
POST /api/index, /index/file, /index/preview	Index synthetic docs, index a real file by server path, or preview its column mapping without indexing
POST /api/collection/create, /delete	Collection lifecycle
POST /api/replica/add, /remove, /api/shard/split	Replica and shard topology changes
POST /api/commit, /optimize, /expunge, /reload, /purge	The five Maintenance-card actions (Section 4.1)
POST /api/tune	Apply one tuning-knob value
POST /api/query	Run a built query (keyword or semantic) and return results, facets, and debug detail
POST /api/model/load, /api/embed	Load an embedding model in the background; embed an existing collection's text field

5.3 Frontend: One HTML File, No Build Step

templates/dashboard.html (2,096 lines) is a single self-contained page — inline CSS, inline vanilla JavaScript, no framework, no bundler, no CDN dependency. It is read from disk once per server process start (a template change needs the dashboard restarted, not just the page reloaded).

The page is organized into independently collapsible sections, in this order:

- **Controls** — load test (collection, RPS slider, start/stop), add documents (synthetic presets or a real file with live column-mapping preview), Collections (existing collections listed with live doc counts and health, plus create/delete), Maintenance (commit, merge, expunge, reload, purge — each described in Section 4.1)
- **Actions** — a scrollable history of what has happened, merging optimistic client-side “started...” notes with the server’s authoritative outcomes; a failed action force-expands the section even if it was collapsed
- **Health** — InsightsEngine findings, current ones shown with the rest foldable, and resolved findings kept (not discarded) in a “resolved earlier” history so a transient spike can’t disappear before it’s read
- **Live signals** — canvas strip charts (5m/15m/1h range) for client- and server-observed latency (with a computed queueing-gap explanation between the two), heap, query rate, GC time share, and segment count per shard; each chart supports hover tooltips
- **Topology** — per-shard replica table with a traffic-light state dot, add/remove replica, shard split, and an expandable per-replica Lucene segment panel (Section 4.8)
- **Cluster activity / Request traffic** — the two LogStream feeds, each with older/newer paging
- **Tuning** — the live knob grid from Section 4.6
- **Query** — the interactive query builder: parser choice (including semantic), filters, facets (added as removable chips, clickable to drill in), an embedding-model selector, and foldable result/debug panels (request URL, parsed query, per-component timing, raw response) — plus a “Run this as a load test” button that hands the built query straight to the load generator, with an on-screen warning if it has no {RAND_*} placeholder to defeat the result cache

State that should survive a page reload but not matter across machines — which sections are collapsed — lives in `localStorage`. Everything else is re-derived from the next `/api/snapshot` poll, which runs on a fixed interval regardless of what the user is doing.

6. Data Flow Walkthroughs

Load test, started from the dashboard

- Browser POSTs `/api/load/start` with a collection and target RPS
- `ActionRunner` validates, builds a `LoadControl`, and submits `loadtest.run_load()` as a coroutine on its background `asyncio` loop — the HTTP handler thread returns immediately
- `run_load`'s scheduler ticks on wall-clock time, reading `control.rps` fresh each tick (so a later slider drag changes the live rate) and writing rolling stats to `.searchlab/live-load.json` once a second
- Every request also flows through `docker`'s `stdout`, which `LogStream` picks up and `loglex.parse_request()` turns into a structured traffic row — so the query section's Request Traffic panel shows the actual queries the load test is firing, not a re-simulation
- The next `/api/snapshot` poll folds `live-load.json` into the response; the browser's chart code appends a point and redraws

Query built and run from the dashboard

- Browser loads `/api/fields` (Luke field list) when a collection is selected, so the parser/facet pickers offer real field names
- “Run query” POSTs the form to `/api/query`
- `query.build_params()` shapes it into Solr params — or, if semantic mode is on, calls the loaded `embeddings.Embedder` to turn the query text into a vector and rewrites `q` into a `{!knn}` clause
- `query.run_query()` executes it, flattens facets, and — with Explain checked — extracts parsed-query, timing, and score-explanation detail from Solr's debug block
- The response renders as one expandable record per document plus foldable panels for the request URL, parsed query, timing breakdown, and raw JSON — nothing the response carries is discarded

Turning a query into a load-test workload

- “Run this as a load test” packages the same query-builder form and POSTs it to `/api/load/start` as the query field
- `ActionRunner.start_load()` calls `query.build_params()` once to validate and shape it, wraps the result as a single-template load list, and passes it to `run_load()` via its `templates=` argument — bypassing the YAML-file query-template path entirely
- If the query text has no `{RAND_WORD}/{RAND_INT}` placeholder, the UI warns that Solr's `queryResultCache` will serve every repeat from cache after the first hit, flattering the latency numbers

7. Cluster State & Configuration

`.searchlab/` (created by `searchlab up`, gitignored) is the only place cluster state lives on disk:

File	Contents
<code>docker-compose.yml</code>	Rendered once from <code>templates/docker-compose{,-es}.yaml.j2</code> via Jinja, from the <code>ClusterSpec</code> — engine, version, node count, heap, GC flags, monitoring toggle
<code>spec.json</code>	The <code>ClusterSpec</code> serialized, read by <code>load_spec()</code> at the start of almost every command that isn't <code>up</code>
<code>live-load.json</code>	Rolling stats written once/second by an in-progress load test; the bridge between the CLI/dashboard's async run and anything polling from outside it
<code>gc-logs/</code>	Present only with <code>--gc-logs</code> ; consumed by <code>searchlab gclog</code>

Because `WORKDIR` is resolved as `Path.cwd() / ".searchlab"` at import time, every command — CLI or dashboard — must be run from the same directory `up` was run from. This is the one piece of implicit state the tool relies on.

8. CLI Command Reference

Command	Does
<code>doctor</code>	Preflight: docker, compose plugin, ports, disk space
<code>up / down / status</code>	Bring the cluster up or down; live node and collection health
<code>create-collection / delete-collection</code>	Collection lifecycle
<code>gen</code>	Generate synthetic JSONL from a data profile
<code>index</code>	Bulk-index a file — JSONL, CSV, TSV, or JSON (see 4.3)
<code>load</code>	Open-loop load test, with <code>--assert</code> regression gates
<code>quickstart</code>	<code>up</code> + <code>create-collection</code> + <code>gen</code> + <code>index</code> + <code>load</code> , one command
<code>chaos kill/pause/unpause/start/restart</code>	Fault injection on one node
<code>chaos run</code>	Timed fault steps from a scenario file
<code>drill</code>	Full orchestrated load + chaos + metrics drill, one HTML report
<code>sweep</code>	One workload across a config matrix, fresh cluster per cell
<code>dashboard</code>	Launch the control panel (<code>--demo</code> for synthetic preview data)
<code>schema</code>	Explicit Solr fields from a data profile (<code>--dry-run</code> to preview)
<code>replay</code>	Replay a real Solr request log at original/scaled/fixed pace
<code>gclog</code>	GC pause analysis from a unified-logging GC log
<code>k8s</code>	Export a Kubernetes operator manifest for the current cluster
<code>metrics / metrics-diff</code>	Snapshot node metrics; diff two snapshots
<code>compare / report-html</code>	Diff two JSON reports; render one as standalone HTML
<code>recall</code>	Recall@k for vector search against brute-force ground truth
<code>learn</code>	Run an interactive lesson against the live cluster
<code>explain</code>	Plain-language translation of a query's debug output

9. Testing Strategy

23 files, 216 tests, run with `pytest -q`, none of which require a Docker daemon. Two patterns carry almost everything:

- **An in-process aiohttp mock server** stands in for a Solr node's HTTP surface — used throughout to assert exact request parameters (e.g. that `expunge` sends `expungeDeletes=true` and not a full `optimize=true`; that a tuning knob writes to the right `Config-API` path; that `shard-fan-out` and `leader-replication` traffic is correctly flagged internal and excluded from the traffic panel)
- **Stub collaborators** for anything slow or external — a deterministic `StubEmbedder` for semantic-search tests, so the suite never downloads model weights to check that a `kNN` query is shaped correctly

Beyond unit coverage, most non-trivial features in this codebase were also verified against a real running cluster before being considered done — indexing real documents and confirming the typed fields are genuinely queryable, splitting a shard and confirming post-split document routing sums correctly, comparing keyword vs. semantic search results on a hand-built catalog where the correct answer shares no words with the query, and reading Solr's own `/admin/segments` output to confirm a metric wasn't stale before charting it.

10. Design Principles

- **Open-loop, always.** Every load-generating path fires on schedule and counts what it couldn't send, rather than silently slowing to match the server — the whole point is to see the incident, not hide it.
- **State lives in the cluster, not in the tool.** No database, no server-side session store beyond one small JSON file and in-memory run state that's fine to lose on restart.
- **Nothing is silently assumed to have worked.** Tuning knobs never touch solrconfig.xml (the Config API writes an overlay instead), so every knob links to the Solr endpoint that proves the value actually changed. A metric that looked suspiciously uniform (segment count pinned at exactly 4 across every shard) was checked against Solr's own admin endpoint before being trusted in a chart.
- **One interface, several engines.** Solr, Elasticsearch, and OpenSearch differ in request shape, response shape, and admin API — engines.py is the only place that's supposed to know that.
- **The UI teaches the API it wraps.** The query builder shows the exact request URL it sent and Solr's real debug output, rather than abstracting them away — the dashboard's job is to make the engine's behavior legible, not to replace understanding it.
- **History over ephemera.** Both the Health panel and the Actions log deliberately keep resolved/past items (foldable) rather than letting them vanish, because a spike that clears between two polls is exactly the kind of thing worth having actually seen.

Generated from the repository at github.com/patrykm10/searchlab.